

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA0615	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	V.LASYA	Reg. No.:	192524186
Section / Batch:	B.TECH AI&DS	Date:	31-08-2026

Instructions to Students

- Answer ALL questions with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big- Θ , Big- Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions	Marks
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP Complete.	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – Pseudocode Writing Task

Q1 Approximation and Tractability

50 marks

Hamiltonian Path Problem

Write a pseudocode using Backtracking to determine whether a Hamiltonian Path exists in a given graph. Clearly define inputs (graph representation) and output (valid path or failure). Design a step-by-step algorithm that ensures each vertex is visited exactly once. Use control structures such as loops and conditional checks for adjacency validation. Incorporate pruning to eliminate invalid paths early. Present the pseudocode in a structured and readable format. Ensure the solution handles all cases efficiently.

Q1. HAMILTONIAN PATH PROBLEM

Problem Statement

Write a pseudocode using backtracking to determine whether a Hamiltonian Path exists in a given graph. A Hamiltonian Path is a path that visits every vertex exactly once. The algorithm should clearly define the graph representation as input and return a valid path or failure as output.

Algorithmic Approach

The problem is solved using the Backtracking technique.

The algorithm constructs the path one vertex at a time. Starting from a selected vertex, it tries to add an unvisited adjacent vertex to the path. Before adding a vertex, it checks whether the vertex is adjacent to the previously selected vertex and whether it has already been visited.

If a valid vertex can be added, the algorithm recursively continues with the next position. If no valid vertex can be added, the algorithm removes the previously selected vertex and tries another possibility.

Input and Output

Input:

- $G \rightarrow$ Graph with V vertices, represented using an adjacency matrix or adjacency list.
- $V \rightarrow$ Number of vertices in the graph.

Output:

- A Hamiltonian Path containing all vertices exactly once, if one exists.
- Failure message if no Hamiltonian Path exists.

Pseudocode

ALGORITHM HAMILTONIAN_PATH(G, V)

INPUT:

$G \rightarrow$ Graph with V vertices

$V \rightarrow$ Number of vertices

OUTPUT:

A Hamiltonian Path if one exists
Otherwise, report failure

```
BEGIN
  path[1...V]  $\leftarrow$  -1
  visited[1...V]  $\leftarrow$  FALSE
  FOR start  $\leftarrow$  1 TO V DO
    path[1]  $\leftarrow$  start
    visited[start]  $\leftarrow$  TRUE

    IF HAM_PATH(2) = TRUE THEN
      PRINT "Hamiltonian Path:"
      PRINT path
      RETURN
    END IF

    path[1]  $\leftarrow$  -1
    visited[start]  $\leftarrow$  FALSE
  END FOR

  PRINT "No Hamiltonian Path exists"
END

FUNCTION HAM_PATH(position)

  BEGIN
    IF position > V THEN
      RETURN TRUE
    END IF

    FOR vertex  $\leftarrow$  1 TO V DO
      IF IS_SAFE(vertex, position) = TRUE THEN

        path[position]  $\leftarrow$  vertex
        visited[vertex]  $\leftarrow$  TRUE
```

```
IF HAM_PATH(position + 1) = TRUE THEN  
RETURN TRUE  
END IF
```

```
path[position] ← -1  
visited[vertex] ← FALSE  
END IF  
END FOR
```

```
RETURN FALSE  
END
```

```
FUNCTION IS_SAFE(vertex, position)
```

```
BEGIN  
IF visited[vertex] = TRUE THEN  
RETURN FALSE  
END IF
```

```
IF position = 1 THEN  
RETURN TRUE  
END IF
```

```
previous ← path[position - 1]
```

```
IF G[previous][vertex] = 1 THEN  
RETURN TRUE  
ELSE  
RETURN FALSE  
END IF  
END
```

Logical Explanation

The algorithm attempts to construct a path that contains every vertex exactly

once. First, each vertex is considered as a possible starting vertex. Once a starting vertex is selected, the algorithm marks it as visited.

For every next position in the path, the algorithm checks all vertices. A vertex can be selected only if it has not been visited and there is an edge between it and the previously selected vertex.

After selecting a valid vertex, the algorithm recursively tries to fill the next position. If the algorithm reaches a position greater than V , all V vertices have been successfully included exactly once, so a Hamiltonian Path has been found. If no valid vertex can be selected at some position, the algorithm backtracks by removing the previously selected vertex and tries another possible vertex.

Example

Consider a graph with vertices A, B, C, D, and E.

Suppose the graph contains the path:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

This path visits every vertex exactly once. Therefore, it is a Hamiltonian Path. The algorithm searches for such a path systematically using inclusion, validation, recursion, and backtracking.

State-Space Representation

The state-space tree represents the different possible choices of vertices at each position of the path.

- Level 1 $\rightarrow$ Starting vertex
- Level 2 $\rightarrow$ Second vertex in the path
- Level 3 $\rightarrow$ Third vertex in the path
- Level 4 $\rightarrow$ Fourth vertex in the path
- Level $V \rightarrow V$ -th vertex in the path

Each node represents a partial Hamiltonian path. A branch is expanded only when the selected vertex is unvisited and is adjacent to the previously selected vertex.

Recursion Steps

1. Choose a starting vertex.
2. Mark the starting vertex as visited.
3. Check all unvisited vertices for the next position.

4. Verify that the selected vertex is adjacent to the previous vertex.
5. Add the valid vertex to the path and mark it as visited.
6. Recursively call the function for the next position.
7. If all V vertices are included, return TRUE.
8. If no valid vertex is available, undo the last selection.
9. Mark the removed vertex as unvisited and try another vertex.
10. If every starting vertex and every possible choice fails, report that no Hamiltonian Path exists.

Pruning Conditions

Invalid paths are eliminated as early as possible using the following

conditions: IF visited[vertex] = TRUE

RETURN FALSE

IF $G[\text{previous}][\text{vertex}] = 0$

RETURN FALSE

The first condition prevents a vertex from being visited more than once. The second condition prevents the algorithm from selecting a vertex that is not directly connected to the previous vertex. These checks prune invalid branches before they are explored further.

Correctness

11. The algorithm considers every possible starting vertex.
12. At each position, only unvisited vertices are considered.
13. A vertex is added only when an edge exists between it and the previous vertex.
14. Therefore, every constructed partial path is valid with respect to adjacency and contains no repeated vertex.
15. When all V vertices are placed in the path, the path visits every vertex exactly once and is therefore a Hamiltonian Path.
16. If a partial path cannot be completed, backtracking removes the last vertex and explores another possibility.
17. Since all possible valid choices are systematically considered, the algorithm finds a Hamiltonian Path whenever one exists.

Complexity Analysis

Let V be the number of vertices in the graph.

At each position, the algorithm may try up to V vertices. In the worst case, it can explore permutations of vertices. The number of possible vertex orderings is $O(V!)$.

Time Complexity: $O(V!)$

The recursion stack, path array, and visited array require $O(V)$ auxiliary space. Auxiliary Space Complexity: $O(V)$

If an adjacency matrix is used, the graph itself requires $O(V^2)$ space.

Complexity Class

The Hamiltonian Path Decision Problem is NP-Complete. Finding a Hamiltonian Path in a general graph is computationally difficult, and the backtracking approach may require exponential or factorial time in the worst case.

Conclusion

The Hamiltonian Path Problem can be solved using backtracking by constructing the path one vertex at a time. The algorithm checks adjacency and visited conditions before adding a vertex and backtracks whenever a partial path cannot be extended. The pruning conditions eliminate invalid paths early. The worst case time complexity is $O(V!)$, while the auxiliary space complexity is $O(V)$, excluding the graph representation.

Q2 Approximation and Tractability

50 marks

Subset Generation Problem with Size Constraint

Write a pseudocode using Backtracking to generate all subsets whose size is exactly k . Clearly define inputs (set of elements, value k) and output (all subsets of size k). Design an algorithm that recursively selects elements while tracking the current subset size. Use loops and recursive calls effectively to explore valid selections. Apply pruning when the current subset size exceeds k or when too few elements remain to complete the subset. Present the pseudocode in a clean and structured format. Ensure all valid subsets are generated completely and without duplication.

Q2 – APPROXIMATION AND TRACTABILITY

SUBSET GENERATION PROBLEM WITH SIZE CONSTRAINT

1. Problem Statement

Given a set of elements S and an integer k , generate all possible subsets of S

whose size is exactly k . The solution uses Backtracking to recursively decide whether each element should be included or excluded. The algorithm must track the current subset size and apply pruning when a partial solution cannot possibly produce a valid subset of size k .

2. Algorithmic Approach

The problem is solved using Backtracking. Starting from the first element, the algorithm explores two choices for every element: include it in the current subset or exclude it. A recursive call is then made for the next element.

Whenever the current subset contains exactly k elements, it is recorded as a valid answer. Branches are pruned when the subset already contains more than k elements or when the remaining elements are insufficient to reach size k .

3. Input and Output

Input:

- S – A set (or array) containing n elements.
- k – The required size of each subset.

Output:

- All unique subsets of S containing exactly k elements.

4. Complete Pseudocode

ALGORITHM GenerateSubsets(S, k)

INPUT: Set S with n elements, integer k

OUTPUT: All subsets of S having exactly k elements

result $\leftarrow$ empty list

currentSubset $\leftarrow$ empty list

$n \leftarrow \text{size}(S)$

PROCEDURE Backtrack(index, currentSubset)

IF $\text{size}(\text{currentSubset}) = k$ THEN

Add a copy of currentSubset to result

RETURN

END IF


```
IF index = n THEN  
RETURN  
END IF
```

```
remaining  $\leftarrow$  n - index
```

```
IF size(currentSubset) + remaining < k THEN  
RETURN  
END IF
```

```
// Include S[index]  
Add S[index] to currentSubset  
Backtrack(index + 1, currentSubset)  
Remove the last element from currentSubset
```

```
// Exclude S[index]  
Backtrack(index + 1, currentSubset)
```

```
END PROCEDURE
```

```
Backtrack(0, currentSubset)  
RETURN result  
END ALGORITHM
```

5. Logical Explanation

- Start with an empty subset and begin at index 0.
- For each element, make two recursive choices: include the element or exclude it.
- When the current subset size becomes exactly k, store or print that subset. • If all elements have been considered and the subset size is still less than k, stop that branch.
- If the number of elements remaining is too small to reach k, prune the branch immediately.
- Because every element is considered in a fixed order, each valid subset is generated once without duplication.

6. Example

Let $S = \{A, B, C, D\}$ and $k = 2$.

The generated subsets are:

$\{A, B\}, \{A, C\}, \{A, D\}, \{B, C\}, \{B, D\}, \{C, D\}$

Therefore, the total number of valid subsets is $C(4, 2) = 6$.

7. State-Space Representation

The state-space tree has one level for each element of S . At every level, the algorithm branches into two possibilities: Include or Exclude. A path from the root to a node represents the decisions made for the elements processed so far. Nodes representing subsets of size k are solution nodes.

$\{\}$

$/ \backslash$

Include A Exclude A

$\{A\} \{\}$

$/ \backslash / \backslash$

Include B Exclude B Include B Exclude B

$\{A, B\} \{A\} \{B\} \{\}$

✓

8. Recursion Steps

1. Call `Backtrack(0, {})`.
2. Choose whether to include the first element.
3. Move recursively to the next index.
4. Continue adding or skipping elements.
5. When the subset reaches size k , record it and return.
6. Backtrack by removing the most recently included element.
7. Explore the exclude branch and continue until all valid combinations are generated.

9. Pruning Conditions

The algorithm stops exploring a branch under the following conditions:

- Valid solution reached: $\text{size}(\text{currentSubset}) = k$. The subset is stored and the branch returns.

- No more elements: $\text{index} = n$ and a subset of size k has not been formed.
- Insufficient remaining elements: $\text{size}(\text{currentSubset}) + (n - \text{index}) < k$. Even if every remaining element is selected, the required size cannot be reached.

10. Correctness

Every subset of S can be represented by a unique sequence of include/exclude decisions for its elements. The backtracking procedure systematically explores these decisions in order. Whenever exactly k elements have been selected, the corresponding subset is produced. Pruning removes only branches that already form a complete solution or cannot possibly reach size k , so no valid subset is lost. Hence, all and only the subsets of size k are generated without duplication.

11. Complexity Analysis

In the worst case, backtracking explores the include/exclude decision space, which is $O(2^n)$. However, the number of required outputs is $C(n, k)$, and copying or printing each subset takes $O(k)$. Thus, the output-related time is $O(C(n, k) \times k)$. The auxiliary recursion stack and current subset require $O(n)$ space in the worst case, excluding the space needed to store all generated subsets.

12. Complexity Class

The search space is exponential in the worst case, $O(2^n)$, but pruning avoids exploring branches that cannot lead to a valid subset of size k . The algorithm is output-sensitive because generating $C(n, k)$ subsets necessarily requires time proportional to the amount of output.

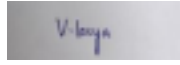
13. Conclusion

Backtracking provides a clear and systematic method for generating all subsets of exactly size k . By tracking the current subset and using pruning conditions, the algorithm avoids unnecessary recursive exploration while ensuring that every valid subset is generated exactly

Code of Conduct

I V.LASYA(192524186) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____


RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Question No.	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)	Total Marks
Q1 (40 Marks)						
Q2 (30 Marks)						
Q3 (30 Marks)						

Overall Total (100)	
---------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____

Date: _____	Date: _____	Date: _____
-------------	-------------	-------------